보안 감사 잔여 8건 — 결정 사항 가이드

> *작성일: 2026-03-26 현황: 보안 감사 33건 중 25건(76%) 완료. 남은 8건은 기술적 결정 또는 정책 결정이 필요.*

목차

1. BIZ-003/004 — CUG 콘텐츠 가시성 정책
2. DATA-003 — 조직 계층 순환 방지 DB 트리거
3. DATA-005 — Flyway 마이그레이션 관리 도입
4. SEC-008 — PII 암호화
5. BIZ-005 — GUARDIAN 보호자 역할
6. BIZ-006 — 커뮤니티 모더레이션
7. DATA-002 — Cross-Service 이벤트 전달
8. DATA-001 — 사용자 탈퇴 시 데이터 정리

---

1. BIZ-003/004 — CUG 콘텐츠 가시성 정책

배경: VideoAclService에 CUG(폐쇄형 단체) 콘텐츠 접근 제어가 대부분 구현되어 있으나, 정책적으로 불명확한 부분이 남아있음.

결정 1-1: CUG 단체의 PUBLIC 콘텐츠 비회원 노출

CUG 단체가 `defaultContentVisibility = PUBLIC` 으로 설정한 콘텐츠를 비회원에게 보여줄 것인가?

Option	설명	유의사항
A. 허용 (현재 구현)	CUG 단체의 PUBLIC 콘텐츠는 홍보용으로 누구나 시청 가능	단체가 실수로 PUBLIC 설정 시 민감 콘텐츠 노출 위험. 단체 관리자에게 "이 콘텐츠는 비회원에게도 공개됩니다" 경고 UI 필요
B. 차단	CUG 단체의 모든 콘텐츠는 회원만 시청 가능	홍보/신규 회원 모집이 어려워짐. 단체가 자체 홍보 영상을 외부에 공유할 수 없음
C. 명시적 홍보 플래그	`is_promotional = true`인 콘텐츠만 비회원에게 노출	가장 안전하지만 UI/UX 복잡도 증가. LiveAsset 엔티티에 필드 추가 필요

결정 1-2: ACL 정책 우선순위

Role ACL, Organization ACL, Team ACL이 모두 설정된 경우 어떤 것이 우선인가?

Option	설명	유의사항
A. OR (하나라도 통과하면 허용, 현재 구현)	역할 ACL, 조직 ACL, 팀 ACL 중 하나라도 만족하면 접근 허용	가장 관대한 정책. 예: "코치" 역할이면 어떤 조직이든 접근 가능. 의도하지 않은 접근 가능성
B. AND (모두 만족해야 허용)	모든 ACL 조건을 동시에 만족해야 접근 허용	가장 제한적. 설정이 복잡하고 관리자가 이해하기 어려움
C. 계층적 (Organization > Team > Role)	조직 ACL 체크 → 통과 시 팀 ACL 체크 → 통과 시 역할 ACL 체크	직관적이지만 구현이 복잡. 상위 계층에서 거부되면 하위는 무시

결정 1-3: CUG 상태 변경 시 캐시 무효화

조직이 CUG ↔ 비CUG로 변경될 때 ACL 캐시를 즉시 무효화할 것인가?

Option	설명	유의사항
A. 즉시 무효화 (권장)	OrganizationService의 CUG 변경 메서드에 `@CacheEvict` 추가	간단한 코드 변경. 캐시 전체 삭제 → 일시적 성능 저하 가능

B. TTL 의존	현재 5분 TTL 만료까지 대기	최대 5분간 잘못된 접근 허용/차단 가능. 보안 이슈

결정 1-4: SUBSCRIBERS + MEMBERS_ONLY 복합 정책

구독자이면서 동시에 회원이어야 하는 콘텐츠가 가능한가?

Option	설명	유의사항
A. 단일 정책만 허용 (현재)	DefaultPolicy는 하나만 설정 가능	단순하고 직관적. 복잡한 비즈니스 요건 대응 불가
B. 복합 정책 지원	policy JSONB에 여러 조건 조합 허용	유연하지만 관리 복잡. VideoAclService 로직 대폭 수정 필요

---

2. DATA-003 — 조직 계층 순환 방지 DB 트리거

배경: Organization은 self-referential parent/children 관계. 현재 Java 앱에서만 순환 감지 (HashSet walk-up). 동시 업데이트 시 앱 레벨 검증이 우회될 수 있음.

결정 2-1: DB 레벨 방어 방식

Option	설명	유의사항
A. BEFORE UPDATE TRIGGER + 재귀 CTE	`parent_id` 변경 시 트리거가 재귀 CTE로 순환 감지	PostgreSQL 전용. 트리거 디버깅 어려움. INSERT/UPDATE마다 재귀 쿼리 실행 → 대량 데이터 시 성능 영향. 현재 74개 테이블 규모에서는 무시 가능
B. EXCLUSION CONSTRAINT	PostgreSQL의 제약 조건으로 방지	순환 방지에는 부적합 (EXCLUSION은 범위 충돌 방지용). 적용 불가
C. 앱 레벨 유지 + Advisory Lock	Java에서 `SELECT FOR UPDATE` + advisory lock으로 동시성 제어	DB 트리거 불필요. 모든 변경이 서비스를 통과한다면 충분. 직접 DB 수정 시에는 방어 불가

결정 2-2: 최대 계층 깊이

Option	값	유의사항
A. 5단계 (현재)	본부 → 지부 → 팀 → 하위팀 → 말단	대부분의 스포츠 조직 구조에 충분
B. 3단계	본부 → 지부 → 팀	더 단순하지만 복잡한 조직 구조 지원 불가
C. 무제한	depth 제한 없음	순환 감지만 하고 깊이는 자유. 성능 예측 어려움

---

3. DATA-005 — Flyway 마이그레이션 관리 도입

배경: 현재 `ddl-auto: update` 로 운영. `db/migrations/` 에 V000~V021 SQL 파일이 있지만 Flyway로 실행되지 않음. 프로덕션 배포 전 필수.

결정 3-1: Flyway 도입 시점

Option	설명	유의사항
A. 즉시 도입	모든 서비스에 Flyway 의존성 추가 + 현재 DB 상태를 baseline으로 설정	기존 DB와 migration SQL의 정합 검증 작업이 큼. ddl-auto가 만든 컬럼명/타입이 migration과 다를 수 있음. **V017~V021 마이그레이션이 이미 적용되었는지 확인 필요**
B. 프로덕션 배포 직전	개발 중에는 ddl-auto 유지, 배포 전에 Flyway 전환	개발 속도 유지. 하지만 전환 시점의 작업량이 많아짐

C. 스테이징 환경부터	staging에만 Flyway 적용, dev는 ddl-auto 유지	점진적 전환. 환경 간 스키마 차이 발생 가능

결정 3-2: 롤백 전략

Option	설명	유의사항
A. Forward-only + 수동 롤백 스크립트	각 마이그레이션에 대응하는 `U{version}__undo_xxx.sql`을 별도 관리	Flyway Community(무료)로 가능. 롤백은 수동 실행. 롤백 테스트가 쉽지 않음
B. Flyway Teams (유료) Undo	Flyway Teams 라이선스로 자동 undo 지원	비용 발생. 자동화된 롤백. 라이선스 관리 필요
C. 롤백 대신 Fix-forward	문제 발생 시 새 마이그레이션으로 수정	가장 실용적. 대부분의 팀이 사용하는 방식. 데이터 손실 마이그레이션에는 부적합

결정 3-3: 마이그레이션 실행 방식

Option	설명	유의사항
A. 앱 시작 시 자동 (spring.flyway.enabled=true)	서비스 부팅 시 Flyway가 자동 실행	간편하지만 여러 서비스가 같은 DB 스키마를 마이그레이션하려 하면 충돌. **서비스별 스키마 분리** 필요 (현재 구조에 적합)
B. CI/CD 파이프라인에서 별도 실행	배포 전 단계에서 `flyway migrate` 실행	앱과 마이그레이션 분리. 순서 제어 가능. CI/CD 설정 필요
C. 전용 마이그레이션 서비스	별도 컨테이너/Job이 마이그레이션만 실행	가장 깔끔하지만 초기 구축 비용

---

4. SEC-008 — PII 암호화

배경: User 엔티티의 email, phoneNumber, birthDate, guardianPhone이 평문 저장 중. 개인정보보호법(PIPA) 준수를 위해 암호화 필요.

결정 4-1: 암호화 대상 필드

Option	대상	유의사항
A. 최소 (법적 필수)	phoneNumber, birthDate, guardianPhone	email은 로그인 ID로 사용 → 암호화 시 검색 문제. phone/birth는 검색 빈도 낮음
B. 확장	email, phoneNumber, birthDate, guardianPhone	email 암호화 시 로그인/중복 체크에 결정적 암호화 필요
C. 전체 PII	위 + name, nationality	가장 안전하지만 성능/복잡도 대폭 증가. name으로 검색하는 기능이 있으면 결정적 암호화 필요

결정 4-2: 암호화 방식 (검색 가능성)

Option	설명	유의사항
A. 결정적 암호화 (Deterministic)	같은 평문 → 항상 같은 암호문	`WHERE email = encrypt('test@example.com')` 가능. 빈도 분석 공격에 취약. email 로그인에 적합
B. 확률적 암호화 (Probabilistic)	같은 평문 → 매번 다른 암호문 (IV/nonce 사용)	가장 안전하지만 `WHERE` 절 검색 불가. 전체 복호화 후 필터링 필요 → 성능 저하

C. 하이브리드	email은 결정적, 나머지는 확률적	가장 현실적. email 검색 가능 + 다른 PII는 높은 보안. **권장**

결정 4-3: 키 관리 방식

Option	설명	유의사항
A. 환경변수	`PII_ENCRYPTION_KEY` 환경변수로 32바이트 키 주입	가장 간단. Docker Compose secrets나 CI/CD 변수로 관리. 키 로테이션 시 재배포 필요. **개발/초기 단계에 적합**
B. AWS KMS / GCP KMS	클라우드 KMS로 키 생성/관리	키가 서버에 노출되지 않음. 자동 로테이션. 클라우드 종속성 + 네트워크 지연. 현재 Docker Compose 환경에서는 과도
C. HashiCorp Vault	자체 호스팅 Vault로 키 관리	클라우드 비종속. 운영 복잡도 높음. 별도 Vault 서버 필요. 현재 규모에서는 과도

결정 4-4: 키 로테이션 전략

Option	설명	유의사항
A. 수동 로테이션	키 변경 시 배치로 전체 재암호화	가장 간단. 다운타임 또는 이중 키 기간 필요. 초기에 적합
B. 키 버전닝	암호문에 키 버전 접두사 포함 (`v1:encrypted_data`)	점진적 로테이션 가능. 읽기 시 버전 확인 → 해당 키로 복호화. 쓰기는 항상 최신 키. 복잡하지만 무중단
C. 봉투 암호화 (Envelope)	DEK(Data Encryption Key)를 KEK(Key Encryption Key)로 암호화	KEK만 로테이션하면 DEK 재암호화 불필요. AWS KMS의 기본 패턴. 구현 복잡

---

5. BIZ-005 — GUARDIAN 보호자 역할

배경: User 엔티티에 guardian 필드들(guardianUserId, guardianPhone, guardianConsentAt, guardianOverrideLimit, isMinor)이 존재하나 API 미구현.

결정 5-1: 보호자-미성년자 관계 모델

Option	설명	유의사항
A. 1:1 (현재 구조)	미성년자 1명 = 보호자 1명 (User.guardianUserId)	간단. 공동 양육자나 형제 관계 미지원
B. 1:N	보호자 1명 = 미성년자 N명	별도 테이블 `guardian_relationships(guardian_id, minor_id)` 필요. 한 부모가 여러 자녀 관리 가능
C. N:M	미성년자 N명 = 보호자 M명	관계 테이블 + 역할(주 보호자, 보조 보호자). 가장 유연하지만 복잡. 동의 책임자가 모호해질 수 있음

결정 5-2: 보호자 동의 프로세스

Option	설명	유의사항
A. 앱 내 동의	보호자가 자신의 계정으로 로그인 → 미성년자 연결 요청 승인	보호자도 앱 가입 필수. UX 간편. 본인 확인 미흡 가능
B. SMS 인증	미성년자 가입 시 보호자 전화번호 입력 → SMS 인증코드 → 동의	보호자 앱 가입 불필요. SMS 비용 발생. 실 보호자 확인 가능
C. 본인 인증 연동	PASS, 카카오 인증 등 본인 확인 서비스 연동	법적으로 가장 확실. 외부 서비스 연동 비용/개발. 14세 미만 필수

D. 대면/오프라인	단체 가입 시 현장에서 서면 동의 → 관리자가 시스템에 입력	온라인 자동화 불가. BO에서 관리자가 동의 상태 직접 변경

결정 5-3: 보호자 제한 범위

guardianOverrideLimit 필드의 의미와 보호자가 제한할 수 있는 범위를 정의해야 함.

Option	설명	유의사항
A. 결제 한도만	월 결제 한도 (원 단위)	가장 간단. guardianOverrideLimit = 월 최대 결제액
B. 결제 + 시청시간	결제 한도 + 일일 시청 시간 제한	별도 필드 추가 필요 (dailyWatchLimitMinutes). 시청 시간 추적 로직 필요
C. 종합 제한	결제, 시청시간, 커뮤니티 활동, 채팅 등	제한 정책 테이블 별도 필요. 구현 복잡도 높음

결정 5-4: 미성년자 기준 연령

Option	기준	유의사항
A. 14세 미만	개인정보보호법상 법정대리인 동의 필수 연령	PIPA 준수. 14~18세는 자율 가입 가능
B. 19세 미만	민법상 미성년자	범위가 넓어 보호자 동의 대상이 많아짐
C. 설정 가능	서비스 설정으로 기준 연령 변경 가능	유연하지만 법적 기준과 별도 관리 필요

---

6. BIZ-006 — 커뮤니티 모더레이션

배경: CommunityPost에 reportPost() 엔드포인트가 스텁으로만 존재. 신고/모더레이션 엔티티 및 워크플로우 미구현.

결정 6-1: 모더레이션 방식

Option	설명	유의사항
A. 사후 신고 (Post-moderation)	게시 즉시 공개 → 신고 접수 시 검토	UX 자유로움. 부적절 콘텐츠 일시 노출 위험. 대부분의 커뮤니티 서비스가 사용
B. 사전 승인 (Pre-moderation)	게시물 작성 → 모더레이터 승인 후 공개	콘텐츠 품질 보장. 게시 지연으로 활성도 저하. 소규모 단체에 적합
C. 하이브리드	기본은 사후 신고. 신규 회원이나 경고 이력 있는 회원은 사전 승인	가장 균형. 구현 복잡. 회원별 상태 관리 필요

결정 6-2: 신고 카테고리

어떤 유형의 신고를 지원할 것인가?

Option	카테고리	유의사항
A. 기본	SPAM, ABUSE, INAPPROPRIATE, OTHER	4가지. 간단하지만 분류 정밀도 낮음
B. 확장	SPAM, ABUSE, HARASSMENT, HATE_SPEECH, SEXUAL, VIOLENCE, COPYRIGHT, MISINFORMATION, OTHER	9가지. 세밀한 분류. 모더레이터 교육 필요
C. 자유 입력	카테고리 없이 사유 텍스트만 입력	가장 간단하지만 통계/자동화 어려움

결정 6-3: 모더레이터 권한 범위

Option	설명	유의사항
A. 조직 내 모더레이터	각 단체의 MANAGER/ADMIN만 해당 단체 게시물 관리	단체 자치. 전체 시스템 관리 어려움
B. 시스템 모더레이터	BO 관리자가 전체 게시물 관리	중앙 집중. 인력 필요. 단체 자율성 저하
C. 이중 구조	단체 모더레이터 + 시스템 모더레이터	1차: 단체 자체 관리. 2차: 시스템 관리자 에스컬레이션. **권장**

결정 6-4: 자동 모더레이션

Option	설명	유의사항
A. 없음 (수동만)	모든 신고를 사람이 처리	초기 단계에 적합. 스케일 시 병목
B. 금칙어 필터	등록된 단어/패턴 자동 필터링	간단한 구현. 우회 쉬움. 오탐 가능
C. AI 기반	외부 AI 서비스(Perspective API 등) 연동	정확도 높음. 외부 의존성 + 비용. 한국어 지원 확인 필요

---

7. DATA-002 — Cross-Service 이벤트 전달

배경: InMemoryEventPublisher가 Spring ApplicationEventPublisher를 사용하여 같은 JVM 내에서만 이벤트 전달. 서비스 간 이벤트(구매→권한, 환불→철회 등)가 실제로 동작하지 않음.

결정 7-1: 메시지 브로커 선택

Option	설명	유의사항
A. Redis Streams	현재 사용 중인 Redis 7에 내장된 스트림 기능 활용	추가 인프라 불필요 (이미 Redis 운영 중). Spring Data Redis로 구현. 처리량 한계 (초당 수만 건). 메시지 보장 수준 약함 (Redis 장애 시 유실 가능). **개발/초기 단계에 적합**
B. RabbitMQ	AMQP 기반 메시지 브로커	Docker Compose에 컨테이너 1개 추가. Spring AMQP 지원 우수. 메시지 보장(ack/nack). 처리량 중간. 단순 pub/sub + 라우팅에 강점. **중소규모에 적합**
C. Apache Kafka	분산 이벤트 스트리밍 플랫폼	높은 처리량. 이벤트 소싱, 리플레이 가능. 운영 복잡 (Zookeeper/KRaft). Docker Compose에 3개 컨테이너(Kafka + Zookeeper + Schema Registry). **대규모/이벤트 소싱에 적합**
D. AWS SQS/SNS	관리형 메시지 서비스	운영 부담 없음. AWS 종속. 현재 Docker Compose 환경과 맞지 않음. 클라우드 이전 후 고려

결정 7-2: Outbox 패턴 적용 여부

DB 트랜잭션과 이벤트 발행의 원자성을 보장할 것인가?

Option	설명	유의사항
A. Outbox 패턴	이벤트를 DB outbox 테이블에 저장 → 별도 프로세스가 브로커로 전달	트랜잭션 보장. 구현 복잡. 각 서비스 DB에 outbox 테이블 필요. Debezium CDC 또는 폴링 방식
B. Best-effort 발행	트랜잭션 커밋 후 즉시 브로커에 발행. 실패 시 재시도	간단. 드문 경우 이벤트 유실 가능. 대부분의 경우 충분. **초기에 권장**
C. Saga 패턴	분산 트랜잭션을 보상 트랜잭션으로 관리	가장 견고하지만 매우 복잡. 현재 규모에서는 과도

결정 7-3: 이벤트 스키마 관리

Option	설명	유의사항
A. JSON (현재)	DomainEvent가 이미 JSON 직렬화 지원	스키마 검증 없음. 유연하지만 계약 깨짐 위험

B. JSON Schema	JSON + 스키마 검증	기존 JSON 호환. 검증 추가. 라이브러리 필요
C. Avro/Protobuf	바이너리 직렬화 + 스키마 레지스트리	높은 성능. 스키마 진화 지원. 학습 곡선. Kafka와 잘 맞음

---

8. DATA-001 — 사용자 탈퇴 시 데이터 정리

배경: User.withdraw()가 status만 변경. 5개 스키마에 분산된 사용자 데이터가 정리되지 않음. 개인정보보호법(PIPA) 위반 가능.

선행 조건: DATA-002(이벤트 전달) 해결 또는 대안 방식 결정

결정 8-1: 처리 방식

Option	설명	유의사항
A. 이벤트 기반 (DATA-002 필요)	UserWithdrawnEvent 발행 → 각 서비스가 수신하여 자체 정리	서비스 분리 원칙 유지. DATA-002 선행 필수. 처리 지연 가능
B. 배치 잡	매일 WITHDRAWN 상태 사용자 조회 → 각 스키마 직접 정리	DATA-002 불필요. 별도 배치 서비스 또는 Identity Service에서 직접 실행. **단, 서비스 간 DB 직접 접근은 MSA 원칙 위반**
C. API 호출 체인	Identity Service가 각 서비스의 삭제 API를 순차 호출	DATA-002 불필요. REST 기반. 한 서비스 장애 시 부분 삭제 위험. 보상 로직 필요
D. 하이브리드	즉시(Identity + Commerce) + 배치(Content + Operation + Admin)	핵심 데이터는 즉시 삭제, 활동 기록은 배치로 정리. 현실적 타협안

결정 8-2: 데이터별 처리 방침

각 테이블의 사용자 데이터를 어떻게 처리할 것인가?

스키마.테이블	Option A: 삭제	Option B: 익명화	Option C: 보관	유의사항
identity.users	PII 필드 초기화	email→hash, phone→null	-	계정 자체는 상태만 변경 (재가입 방지용 email hash 보관)
commerce.purchases	-	user_id → -1	5년 보관	**세법상 거래 기록 5년 보관 의무**
commerce.entitlements	삭제	-	-	탈퇴 시 권한 즉시 철회
commerce.wallets	잔액 0으로	-	-	잔액 환불 절차 먼저 필요
content.community_posts	삭제	author_user_id → -1 (글 유지)	-	"탈퇴한 사용자" 표시로 글 유지 vs 완전 삭제
content.memberships	삭제	-	-	단체 멤버 수 갱신 필요
content.follows, likes, comments	삭제	-	-	카운트 필드 갱신 필요
content.view_histories	삭제	-	-	개인 시청 기록
operation.reservations	-	user_id → -1	완료된 예약 보관	진행 중 예약은 자동 취소
admin.audit_logs	-	-	영구 보관	**감사 로그는 법적 보관 의무 가능**

결정 8-3: 유예 기간

Option	기간	유의사항
A. 즉시 삭제	탈퇴 요청 즉시 처리	PIPA 준수 (지체 없이 파기). 복구 불가
B. 30일 유예	탈퇴 후 30일간 복구 가능	사용자 실수 방지. PIPA는 "정당한 사유" 시 유예 허용. 유예 기간 중 데이터 접근 차단 필요
C. 90일 유예	탈퇴 후 90일간 복구 가능	넉넉한 복구 기간. 법적 근거 약함

결정 체크리스트

아래 체크리스트의 결정을 완료하면 해당 항목 구현을 즉시 시작할 수 있습니다.

즉시 진행 가능 (결정 1-2개)

- ☐ **1-3**: CUG 캐시 무효화 → A(즉시 무효화) 선택 시 코드 1줄 추가
- ☐ **2-1**: DAG 트리거 방식 선택 (A/C)
- ☐ **2-2**: 최대 깊이 확정 (5 유지?)

단기 진행 가능 (결정 3-4개)

- ☐ **3-1**: Flyway 도입 시점 (A/B/C)
- ☐ **3-2**: 롤백 전략 (A/B/C)
- ☐ **3-3**: 마이그레이션 실행 방식 (A/B/C)
- ☐ **4-1**: PII 암호화 대상 (A/B/C)
- ☐ **4-2**: 암호화 방식 (A/B/C)
- ☐ **4-3**: 키 관리 (A/B/C)

중기 진행 (비즈니스 정책)

- ☐ **1-1**: CUG PUBLIC 콘텐츠 정책 (A/B/C)
- ☐ **1-2**: ACL 우선순위 (A/B/C)
- ☐ **5-1**: 보호자 관계 모델 (A/B/C)
- ☐ **5-2**: 동의 프로세스 (A/B/C/D)
- ☐ **5-3**: 제한 범위 (A/B/C)
- ☐ **5-4**: 미성년자 기준 연령 (A/B/C)
- ☐ **6-1**: 모더레이션 방식 (A/B/C)
- ☐ **6-2**: 신고 카테고리 (A/B/C)
- ☐ **6-3**: 모더레이터 권한 (A/B/C)

장기 진행 (인프라)

- ☐ **7-1**: 메시지 브로커 선택 (A/B/C/D)
- ☐ **7-2**: Outbox 패턴 (A/B/C)
- ☐ **8-1**: 탈퇴 처리 방식 (A/B/C/D)
- ☐ **8-2**: 테이블별 처리 방침
- ☐ **8-3**: 유예 기간 (A/B/C)